SALENTO DEV · LIGHTNING NIGHT 2026

Same App, Different Stack.

Una micro-app. Quattro framework.
Stesso obiettivo, quattro modi di svilupparla.

React Native | Flutter | Kotlin Multiplatform | Ionic

Alessandro Marra

29 ottobre 2026

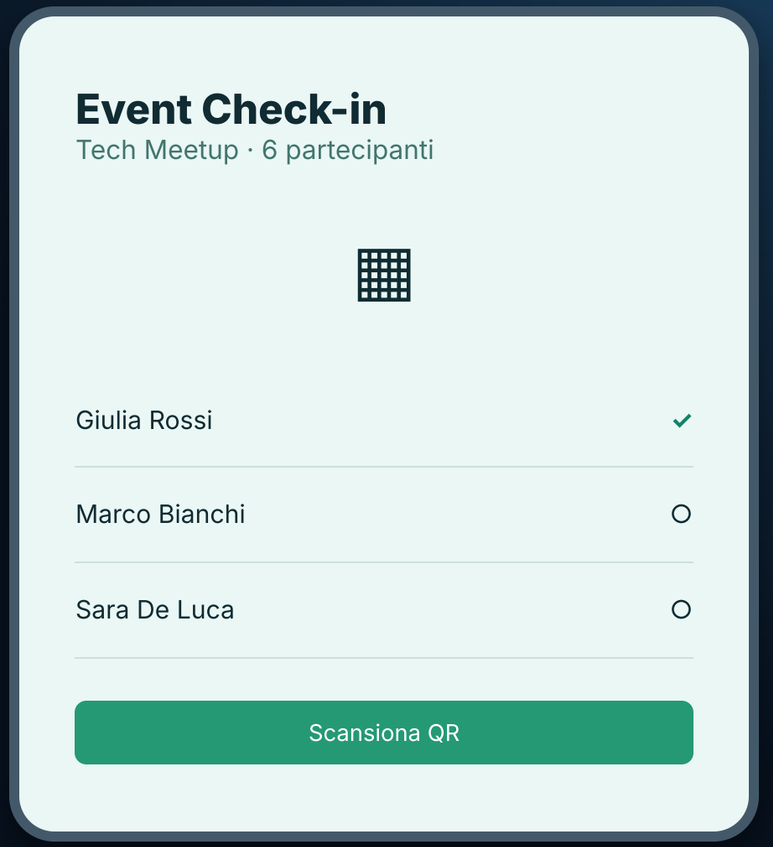
Tutto parte da una richiesta semplice.

> **«Ci serve un'app mobile per gestire il check-in di un evento.»**

EVENT CHECK-IN · REQUISITI ESSENZIALI

- 6 partecipanti e ricerca per nome/ID
- Scansione QR e registrazione check-in
- Persistenza locale e reset
- Stessi dati e stessa esperienza per tutti

Android + iOS richiesti · Prove su Android · Nessun backend



Una domanda. Quattro risposte.

Chiedi quale framework utilizzare e ogni collega ha già il suo preferito.



React Native + Expo

Alessandro

«Ho sempre usato questo. Perché cambiare?»

TypeScript · componenti nativi · ecosistema Expo



Flutter

Matteo

«Ne parlano tutti su LinkedIn.»

Dart · widget e rendering gestiti da Flutter · UI condivisa



KMP + Compose

Ivan

«Kotlin. Serve davvero aggiungere altro?»

Kotlin · logica condivisa e UI Compose opzionalmente condivisa



Ionic + Capacitor

Fabio

«Per tutto il resto, c'è un plugin. Forse.»

HTML/CSS/TypeScript · UI WebView su mobile · plugin nativi

Quattro opinioni, tutte ragionevolissime. Almeno per chi le propone.

Stesso brief. Quattro implementazioni.

Un requisito funzionale comune, istruzioni tecniche specifiche per ciascuno stack.

IL BRIEF COMUNE

Event Check-in · Tech Meetup 2026

- **06** — **6 partecipanti** — Ricerca per nome e ID
- **QR** — **Check-in con QR** — Validi, doppi, sconosciuti
- **Stato persistente** — Inserimento manuale e reset

LE SCELTE TECNICHE

UI · Scanner · Storage

React Native + Expo

UI	Componenti RN
QR	Expo Camera
Dati	AsyncStorage

Flutter

UI	Material 3
QR	mobile_scanner
Dati	shared_preferences

KMP + Compose

UI	Compose condivisa
QR	Scanner di piattaforma
Dati	Adapter per piattaforma

Ionic + Capacitor

UI	Ionic React*
QR	Plugin Capacitor
Dati	Preferences

Stessa AI **Sol 6.1** · effort Alto · Windows · 1 generazione per stack

Prompt originali P

* Ionic React è la scelta di questo esperimento, non un vincolo di Ionic.

Stessa azione. Quattro runtime.

Quattro implementazioni di Event Check-in, mostrate sul dispositivo Android.

React Native + Expo



Release Android · 17,7 s

Flutter



Release Android · 13,3 s

KMP + Compose



Release Android · 12,0 s

Ionic + Capacitor



Release Android · 14,2 s

Video riproducibili nella versione HTML

Fotogrammi dalle registrazioni originali

06 / ESPERIMENTO · GENERAZIONE

Quanto è costato generare il codice?

Sol 6.1 · effort Alto · Windows · una esecuzione per framework · nessuna compattazione

FRAMEWORK	TEMPO AI	CONTESTO FINALE	QUOTA 5H USATA
React Native + Expo	37m 46s	240.337 token	31 p.p.
Flutter	21m 56s	165.275 token	17 p.p.
KMP + Compose	25m 47s	161.054 token	14 p.p.
Ionic + Capacitor	39m 38s	219.815 token	28 p.p.

2h 05m 07s
Tempo totale AI osservato

21m 56s
Generazione più rapida: Flutter

4 / 4
Progetti generati

Contesto finale ≠ token complessivi elaborati. Quota 5h ≠ conteggio token. Test senza repliche: nessuna inferenza statistica generale.

Dal prompt all'APK installabile

Xiaomi M2101K9G · Android 12 / API 31 · output puliti · cache SDK già disponibili

FRAMEWORK	DEBUG: TENTATIVI / TEMPO	RELEASE: TENTATIVI / TEMPO	APK RELEASE
React Native + Expo	2 · 3m 54,74s	3 · 7m 42,63s	29,65 MB*
Flutter	1 · 2m 18,37s	1 · 3m 04,14s	63,82 MB
KMP + Compose	1 · 57,57s	1 · 1m 24,30s	10,99 MB
Ionic + Capacitor	1 · 1m 55,17s	1 · 3m 19,27s	35,75 MB

4/4
QR e persistenza OK in Debug

4/4
Release autonome verificate

3/4
Release al primo tentativo

* RN solo ARM64; gli altri APK universali. R8 disattivato in KMP e Ionic. Tempi dei tentativi cumulati, esclusi setup/download e test; KMP/Ionic comprendono firma separata.

Fin qui, quello che abbiamo misurato. Ora, quello che dichiarano i framework.

Un esperimento ci dice come sono andate **queste quattro implementazioni**. Per scegliere una tecnologia, non basta.

01 · EVIDENZA SPERIMENTALE

Il nostro test

Generazione AI, build, errori e APK.
Un'esecuzione per framework.

02 · FONTI UFFICIALI

Oltre il prototipo

Identità dei framework, adozione, policy di supporto e aggiornamenti.

Due prospettive complementari. Un supporto dichiarato non equivale a un'app già verificata; una singola build non descrive il futuro dello stack.

Quattro stack. Quattro UI diverse.

Linguaggio, rendering ed esempi reali, a colpo d'occhio.

React Native

DAL 2015

+ Expo

JS / TS

RENDERING UI

React › Fabric › Viste native

INGRESSO

Familiarità con React

Discord
app mobile

Flutter

DAL 2018

Dart

RENDERING UI

Widget › Engine › Pixel

INGRESSO

Dart e modello a widget

Google Pay
caso Flutter

KMP

DAL 2017

+ Compose

Kotlin

RENDERING UI

Android: Compose › iOS: Skia / Skiko

INGRESSO

Kotlin + integrazioni native

McDonald's
adozione KMP*

Ionic

DAL 2013

+ Capacitor

JS / TS · HTML / CSS

RENDERING UI

Web Components › WebView

INGRESSO

Competenze Web

Burger King
adozione Capacitor*

Nota: KMP può condividere la logica anche senza Compose. *Adozioni documentate, non necessariamente dell'intera UI.

Curva di ingresso qualitativa: dipende dalle competenze del team · fonti cliccabili nelle card

Cosa dichiarano le fonti ufficiali?

Politiche, release e supporto piattaforme · ottobre 2026 · indicatori diversi, non punteggi

FRAMEWORK	DATO UFFICIALE	CHE COSA SIGNIFICA PER LA MANUTENZIONE	FONTE
React Native + Expo	**~6 minor/anno** 3 serie RN supportate	Upgrade frequenti; allineamento fra RN, SDK Expo e moduli.	RN ↗
Flutter	**4 finestre stable previste** nel 2026 · hotfix esclusi	Release pianificate circa ogni trimestre; verificare migrazioni e plugin.	Calendario ↗
KMP + Compose	**3 target UI stabili** Android · iOS · Desktop	Controllare la compatibilità di Kotlin, Compose, Gradle e librerie.	Stabilità ↗
Ionic + Capacitor	**1 major/anno** Capacitor 8 → API 36	Major legate ai requisiti Android; aggiornare bridge, plugin e Web stack.	Upgrade 8 ↗

Per scegliere conta anche il dopo: politica di supporto, compatibilità dei plugin e prevedibilità degli aggiornamenti.

Indicatori **eterogenei**: non sono confrontabili come un punteggio unico e non quantificano ore di manutenzione né qualità del codice.

Il miglior framework è...

D **Davide**
Messaggio appena ricevuto

Ragazzi, ottimo lavoro! Piccolissima richiesta: il progetto deve funzionare anche sul Web. 😉

Il Web cambia le priorità.

Quanto lavoro serve per far funzionare Event Check-in anche nel browser?

FRAMEWORK	COMPLESSITÀ	MOTIVAZIONE PER EVENT CHECK-IN
React Native + Expo	● Bassa	**Expo Web e `expo-camera` supportano Web e QR.** Verificare build browser, permessi camera e storage.
Flutter	● Bassa	**Flutter Web e `mobile_scanner` supportano il browser.** Testare focus, accessibilità e compatibilità dello scanner.
KMP + Compose	● Media	**UI Compose riutilizzabile, ma Web/Wasm è Beta.** Integrare scanner e storage Web; verificare compatibilità browser.
Ionic + Capacitor	● Bassa	**UI già Web e scanner Capacitor con implementazione browser.** Verificare fotocamera, permessi e comportamento nei browser.

Stima qualitativa per Event Check-in, basata sulle fonti ufficiali: le estensioni delle quattro app originali non sono state testate.

IL PUNTO **Web:** tre percorsi consolidati, uno ancora Beta. La complessità dipende soprattutto dalle integrazioni concrete.

Stime documentali · quattro app originali non testate su Web · Expo · Flutter · KMP · Ionic

Quindi, il miglior framework è...

Davide
Un altro messaggio · proprio ora

Ultimissima cosa, promesso: ci serve anche l'app desktop per Windows e macOS. 😇

I requisiti hanno appena cambiato idea. Ancora.

Desktop: supportato non vuol dire pronto.

Obiettivo: app installabili per Windows e macOS, con QR e persistenza funzionanti.

FRAMEWORK	COMPLESSITÀ	MOTIVAZIONE PER EVENT CHECK-IN
React Native + Expo	Alta	**Windows e macOS richiedono implementazioni React Native dedicate.** Verificare compatibilità Expo, plugin QR e configurazioni native.
Flutter	Media	**Build Desktop ufficiali per Windows e macOS.** `mobile_scanner` supporta macOS ma non Windows: lì serve un QR scanner alternativo.
KMP + Compose	Media	**Compose Desktop/JVM è stabile e la UI è condivisibile.** Servono adattatori per scanner QR e storage, più packaging per i due sistemi.
Ionic + Capacitor	Media	**La UI Web è riutilizzabile.** Per app installabili Windows/macOS servono Electron o Tauri, integrazioni di sistema e packaging.

Stima qualitativa per Event Check-in, basata sulle fonti ufficiali: le estensioni delle quattro app originali non sono state testate.

IL PUNTO **Desktop:** Flutter e Compose offrono target diretti, ma per il check-in contano anche scanner, integrazioni e distribuzione.

Stime documentali · quattro app originali non testate su Desktop · RN · Flutter · KMP · Ionic · QR

15 / RIPARTIAMO DAL BRIEF

E se lo avessimo saputo fin dall'inizio?



Mobile



Web

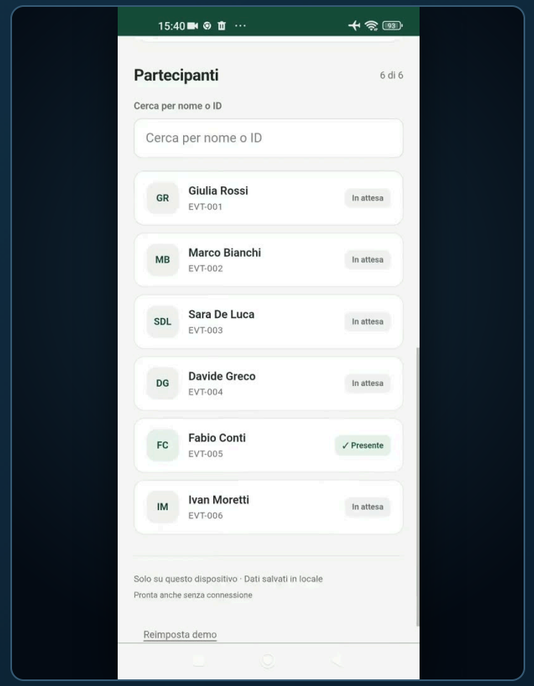


Desktop

Questa volta, sono partito dai requisiti.

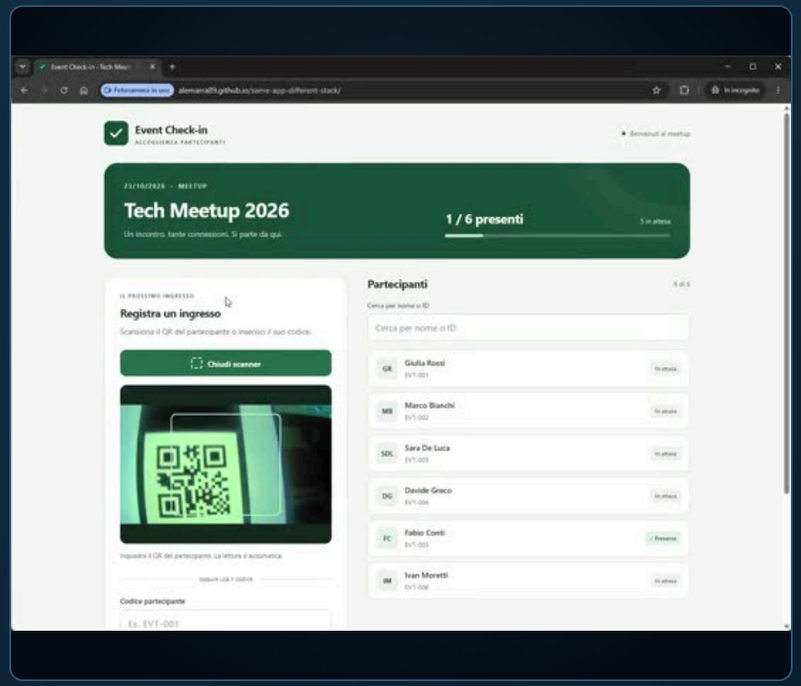
Event Check-in in azione su smartphone, nel browser e sul desktop.

Mobile



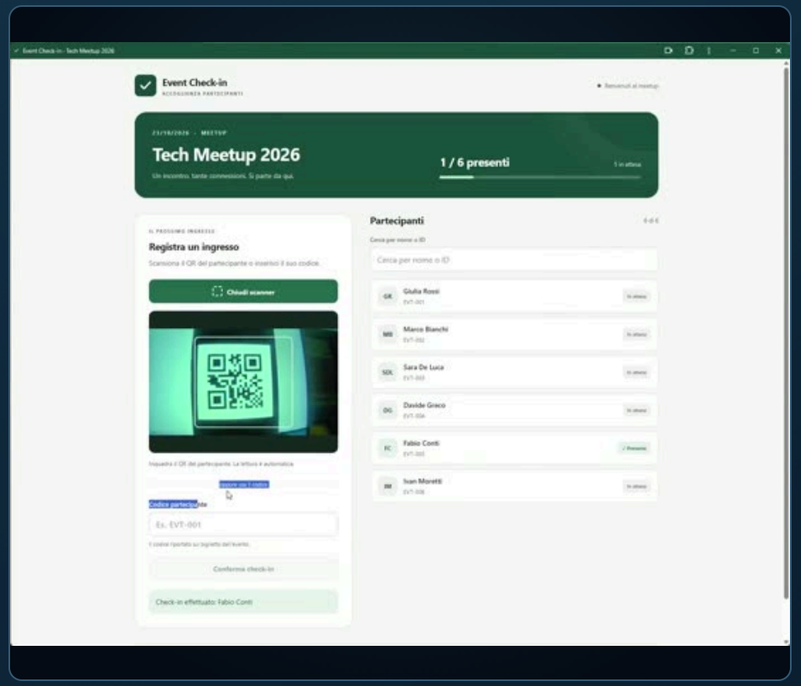
Android · app installata **13 s**

Web



Applicazione nel browser **13 s**

Desktop



App installata · finestra autonoma **22 s**

Video riproducibili nella versione HTML

Fotogrammi dalle registrazioni originali

E ADESSO TOCCA A VOI.

Secondo voi, quale framework ho utilizzato?

Mobile

Web

Desktop

Un'ultima scelta. Una sola domanda.

SAME APP. DIFFERENT STACK.

Nessun framework vince sempre.

Il migliore è quello giusto **per i requisiti**.
E per le persone che useranno e manterranno il prodotto.

Requisiti | Esperienza utente | Team | Community | Ecosistema | Manutenzione

SAME APP. DIFFERENT STACK. · SEE YOU ON GITHUB

Grazie!

Domande? Curiosità? Il codice è a disposizione.



Alessandro Marra

Software Architect @linksmt

Co-founder @salento.dev | Scrum Master | Web Accessibility Expert

linkedin.com/in/alemarra89

github.com/alemarra89

Esplora il progetto

Codice sorgente · esperimento · confronti

github.com/alemarra89/
same-app-different-stack